

codeHive ACADEMY

Radix Sort

DIGIT-BY-DIGIT PRECISION

DSA Series: Module 12

codeHive

1. The Radix Strategy

Radix Sort is a non-comparative sorting algorithm that avoids direct comparisons between numbers. Instead, it groups numbers into buckets based on their individual digits.

It typically starts from the **Least Significant Digit (LSD)**—the rightmost digit—and moves toward the left.

What is a Radix?

The "Radix" or "Base" is the number of unique digits used in a number system. For our standard decimal system, the radix is **10** (digits 0–9). This is why Radix Sort uses 10 buckets.

2. The Necessity of Stability

CRITICAL CONCEPT

Radix Sort **must** use a **Stable** underlying sorting mechanism. A stable sort preserves the relative order of elements that have the same value.

Why? When we move from sorting the "ones" column to the "tens" column, a stable sort ensures we don't accidentally scramble the progress we made on the "ones" column.

The Logic

- Sort by 1s digit.
- Sort by 10s digit (keeping order of 1s).
- Sort by 100s digit (keeping order of 10s & 1s).

codeHive

3. Manual Run-Through

Array: [33, 45, 40, 25, 17, 24]

Pass 1: Focus on Ones Column

40			33	24	45,25	17
0	1	2	3	4	5	7

Resulting Array: [40, 33, 24, 45, 25, 17]

Pass 2: Focus on Tens Column

17	24,25	33	40,45
1	2	3	4

Final Sorted Array: [17, 24, 25, 33, 40, 45]

4. Python Implementation

```
myArray = [170, 45, 75, 90, 802, 24, 2, 66]
radixArray = [[], [], [], [], [], [], [], [], [], []]
maxVal = max(myArray)
exp = 1 # 1s, 10s, 100s ...

while maxVal // exp > 0:
    # Place values into buckets
    while len(myArray) > 0:
        val = myArray.pop()
        radixIndex = (val // exp) % 10
        radixArray[radiusIndex].append(val)

    # Put values back into array from buckets
    for bucket in radixArray:
        while len(bucket) > 0:
            val = bucket.pop()
            myArray.append(val)

    exp *= 10

print("Sorted array:", myArray)
```

5. Analysis & Complexity

$$O(n \cdot k)$$

Where **n** is the number of elements and **k** is the number of digits in the largest number.

When to use Radix Sort?

- **Ideal:** Huge datasets (large n) where numbers have few digits (small k). In this case, efficiency approaches **$O(n)$** .
- **Avoid:** When k is very large (e.g., numbers with as many digits as there are elements). Efficiency drops to **$O(n^2)$** .

6. Knowledge Check

Question: To sort an array with Radix Sort, what property must the underlying sorting have for the final result to be correct?

Answer: Stable Sorting

© 2026 codeHive Academy • Professional Series • Radix Sort Mastery